

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA

CLASE 08 • NIVEL 3 - AVANZADO

Clase 08: Sistemas Multi-Agente, Supervisión y Guardrails

«Una Empresa de Agentes Especializados Coordinados por un Director»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 08** del **Curso 3: Creación y Desarrollo de Agentes de IA**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Comprender la orquestación multi-agente, delegación jerárquica, consenso y capas de guardrails de seguridad.

Competencia Práctica: Construir un sistema con un Agente Investigador, un Agente Redactor y un Agente Supervisor.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 08: Sistemas Multi-Agente, Supervisión y Guardrails

Para problemas complejos, múltiples agentes especializados colaboran mejor que un único agente generalista.

☀ **Metáfora Central: Una Empresa de Agentes Especializados Coordinados por un Director**

Es como una agencia de noticias: el reportero investiga los hechos, el redactor escribe la noticia y el editor jefe revisa la calidad.

Principios Teóricos y Modelo Mental

Patrón Supervisor: Un agente orquestador recibe la tarea global y la desglosa delegando a agentes especialistas.

Guardrails: Filtros deterministas que validan entradas y salidas para prevenir toxicidad, fugas de datos (PII) y alucinaciones.

⚡ **Regla de Oro en Python**

Asigna a cada agente un System Prompt ultra específico y un conjunto reducido de herramientas.

Arquitectura y Movimiento de Datos en Memoria

Orquestación multi-agente con supervisor y validación por guardrails.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Recepción de solicitud y análisis del Supervisor.	Plan de tareas desglosado.
2. Evaluación	Delegación al Agente Investigador (Tool Calling).	Datos crudos recolectados.
3. Transformación	Paso de datos al Agente Redactor para síntesis.	Borrador de reporte generado.
4. Retorno / Salida	Filtro por Guardrails de seguridad y aprobación final.	Entrega final aprobada.

Visualización Mental

Divide y vencerás: agentes pequeños y especializados son más confiables que uno monolítico.

Código de Demostración en Python 3.10+

Orquestador multi-agente colaborativo en Python:

main.py (Python 3.10+)

PEP 8 Compliant

```
class MultiAgentSystem:
    def __init__(self):
        pass

    def agente_investigador(self, tema: str) -> dict:
        return {"datos": f"Hallazgos clave sobre {tema}: Crecimiento del 40% en adopción."}

    def agente_redactor(self, investigacion: dict) -> str:
        return f"Reporte Ejecutivo: {investigacion['datos']}"

    def supervisor(self, tema: str) -> str:
        print("👑 Supervisor: Coordinando equipo...")
        datos = self.agente_investigador(tema)
        informe = self.agente_redactor(datos)
        return f"✅ Publicación Aprobada:\n{informe}"

sistema = MultiAgentSystem()
print(sistema.supervisor("Agentes Autónomos en 2026"))
```

Análisis de Ingeniería del Código

Separación de roles en métodos independientes y flujo orquestado por el supervisor.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Fricción de comunicación entre agentes.

⚠ Gotcha Frecuente (Trampa de Principiante)

Pasar texto libre desordenado entre agentes provoca pérdida de contexto en cadenas largas.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
msg_agente_2 = call_llm(f'El otro dijo:
{texto_libre_caotico}') # ❌ Degradación
```

✅ Patrón Correcto

Usa esquemas Pydantic para el paso de mensajes entre agentes ✅

🛡 Consejo de Resiliencia en Producción

Implementa un historial de mensajes estructurado compartido en el estado del grafo.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 08: Sistemas Multi-Agente, Supervisión y Guardrails**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Una Empresa de Agentes Especializados Coordinados por un Director
2. Regla de Oro	Asigna a cada agente un System Prompt ultra específico y un conjunto reducido de herramientas.
3. Gotcha Crítico	Pasar texto libre desordenado entre agentes provoca pérdida de contexto en cadenas largas.
4. Buenas Prácticas	Implementa un historial de mensajes estructurado compartido en el estado del grafo.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Diseña un sistema con un Agente Programador y un Agente Revisor de Código que valide pruebas unitarias.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.